

오늘의 작업 기록 — 2026년 6월 20일

Java 학습 시작 + mini-ledger 프로젝트 **중분 1 (Account 클래스) 완성** 어제(6/19) = 개발 환경 구축 / **오늘(6/20) = 첫 진짜 코드** (Hello World 다음 단계) 작업 환경: 아이폰 + Termius(SSH) + DigitalOcean Ubuntu 24.04 VPS / nano · javac · git · gh

한눈에 — 오늘 한 4가지

- Java 학습 방법론 재조정 — 제로베이스 인정, 기초부터
- 학습 자료 매핑 — 점프 투 자바 챕터 ↔ 과제 중분 연결
- 중분 1: Account 클래스 작성 — deposit / withdraw 직접 구현(★오늘의 핵심★)
- mini-ledger repo 생성 + 첫 커밋/푸시 — 잔디

결과: Hello World만 하던 제로베이스에서 → 내가 짠 클래스가 컴파일되고 공개 repo에 올라감.

Phase 1 — 학습 방법론 정립

상황: 어제 환경은 깔끔했지만, 오늘 Account 코드를 읽지도 못함. Hello World(한 줄) → 클래스 전체로 점프한 게 무리였음. → 한 칸 내려가 기초부터 가기로.

정한 것 — 지속 가능한 학습 시스템 3개: - (A) 구조화된 기초를 순서대로 — 점프 투 자바를 변수 → 메서드 → 클래스 순으로, 예제를 직접 타이핑하며. (강의 정주행 X, 만들며 배우되 기초는 먼저.) - (B) 직접 만들며 적용 — 코드는 내 손으로 짠다. - (C) 마힘 프로토콜 — ① 자료 재읽기 → ② 에러 읽고 검색 → ③ 그래도 안 되면 질문(답 아닌 힌트).

원칙: - 읽기(인식) ≠ 쓰기(실력). 시가 다 짜주면 면접 꼬리질문에서 무너짐. - productive struggle(막혀 끄공대는 것)이 곧 학습. - 목표 궤적: 시간 갈수록 도움을 덜필요로 함 = 자립.

Phase 2 — 학습 자료 매핑 (점프 투 자바)

중분 ↔ 읽을 챕터:

중분	만들 것	챕터 (쪽)
1	Account	05-02 클래스(140~147), 05-03 메서드(148~158)
2	Bank	03-07 리스트(75~84), 03-08 맵(85~89)
3	커스텀 예외	07-04 예외 처리(241~252)
4	enum	03-10 상수 집합(97~100)

기초(중분 전 필수): 02-02 변수(36~40), 03-01 숫자(46~51), 04-01 if문(111~118).

PC 교재 → 폰+VPS 적용: - 책은 인텔리제이(IDE) 전제 → 설치 챕터는 무시. 나는 이미 VPS+nano+javac. - 자바 코드 자체는 환경 안 탐 — 책 예제 그대로, "인텔리제이 실행"만 javac X.java && java X 로 번역. - 배운 것: 자바는 어디서나 같은 자바. IDE는 편의일 뿐, 자바가 아님.

Phase 3 — 중분 1: Account 클래스 작성 ★오늘의 핵심★

레슨 0 — 변수 (값을 담는 상자)

```
long balance = 1000;
balance = balance + 500;    // → 1500
System.out.println(balance);
```

- long balance = 1000; = " balance 라는 상자(long =정수)에 1000 넣기"
- balance = balance + 500; = "상자 값을 (현재값 + 500)으로 바꿔 다시 넣기"
- deposit의 balance = balance + amount 가 이것과 똑같은을 이해. (어제 안 읽히던 그 줄)

Account 구조 이해

- public class Account { } = 계좌 설계도(클래스)
- 필드 id (고유번호) · owner (주인) · balance (잔액)
- private = 캡슐화(밖에서 직접 못 건드림), final = 재할당 불가
- 생성자 = 계좌 만들 때 필드 채움
- getBalance() = 잔액 읽기 통로
- deposit / withdraw = 내가 채울 두 메서드

deposit 구현 — 직접 작성

```
public void deposit(long amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("입금액은 0보다 커야 합니다");
    }
    balance = balance + amount;
}
```

한 줄씩: - `if (amount <= 0) { ... }` → 0 이하 금액 막기 (나쁜 입력 차단) - `throw new IllegalArgumentException(...)` → 에러 던지고 메서드 멈춤 - `balance = balance + amount;` → 잔액에 더해 다시 저장

withdraw 구현 — 직접 시도 → 교정

처음 시도에서 잡은 실수 3개 (← 진짜 배운 부분): 1. 메서드 이름을 또 `deposit` 으로 씀 → 같은 이름 중복 금지, 빼기니까 `withdraw` 2. 설명을 코드에 붙여넣음 — "상태가 안 됨" = 잔액 부족에 딱 은 `IllegalStateException` 이 뭔지 설명한 말이지 코드가 아님. (교훈: 설명 ≠ 코드) 3. `amount <= 0` 검사를 빠뜨림 — 두 검사 다 필요

완성:

```
public void withdraw(long amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("출금액은 0보다 커야 합니다");
    }
    if (balance < amount) {
        throw new IllegalStateException("잔액이 부족합니다");
    }
    balance = balance - amount;
}
```

한 줄씩: - 검사 1: `amount <= 0` → deposit과 동일 (음수 출금 막기) - 검사 2: `balance < amount` → 잔액 부족이면 막기 (`IllegalStateException` = "상태가 안 됨") - `balance = balance - amount;` → 통과 후 빼기 (deposit은 `+`, 이걸 `-`)

핵심 패턴: 검사 두 개 먼저 → 통과 후에야 balance 변경. = 과제의 "바꾸기 전에 검증" 원칙. 잘못된 출금이면 빼기 전에 막아 잔액 보호 → 나중의 ACID·트랜잭션 사고의 씨앗.

적용한 결제 엔지니어 원칙

- 돈은 `long` (double 금지 — 부동소수점 = 돈 버그)
- 검증 먼저 (상태 바꾸기 전 검사)
- 도메인 실패 = 예외 (그냥 진행 X)

파일 생성 + 컴파일 (여기서 환경 개념도 배움)

- 셸 vs 에디터 구분: `cat` · `javac` · `git` 은 터미널 명령, nano 안이 아님. (cat 명령을 nano에 넣으려다 교정.)
- 디렉토리: `~` = 홈, 프로젝트는 `~/mini-ledger` 폴더. `cd` 로 이동. (프롬프트 `~/mini-ledger#` 로 바뀌는 걸로 확인)
- cat heredoc으로 파일 통째 생성: `bash mkdir -p ~/mini-ledger && cd ~/mini-ledger cat > Account.java << 'EOF' ... (전체 코드) ... EOF ( EOF ~ EOF 사이를 Account.java 내용으로 통째 덮어씀. 중복·위치 걱정 없이 한 방.)`
- 컴파일: `javac Account.java` → `Account.class` 생성 = 에러 0, 컴파일 성공. (`ls` 로 `Account.class` 확인)

완성된 Account.java (전체)

```

public class Account {
    private final String id;
    private final String owner;
    private long balance;

    public Account(String id, String owner, long initialBalance) {
        this.id = id;
        this.owner = owner;
        this.balance = initialBalance;
    }

    public long getBalance() {
        return balance;
    }

    public void deposit(long amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("입금액은 0보다 커야 합니다");
        }
        balance = balance + amount;
    }

    public void withdraw(long amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("출금액은 0보다 커야 합니다");
        }
        if (balance < amount) {
            throw new IllegalStateException("잔액이 부족합니다");
        }
        balance = balance - amount;
    }
}

```

Phase 4 — 커밋 + GitHub (잔디)

```

git init
echo "*.class" > .gitignore
git add .
git commit -m "implement Account with deposit/withdraw"
gh repo create mini-ledger --public --source=. --remote=origin --push

```

- `git init` → mini-ledger를 git 관리 시작
- `.gitignore` → `Account.class` (빌드 산물) 커밋 제외, *소스만*
- 커밋: `9e671d6` — "implement Account with deposit/withdraw" (2 files changed, 33 insertions)
- `gh repo create` *한 줄로* → GitHub repo 생성 + remote 연결 + push *한 방에* (Safari 0회). *어제 깐 gh의 위력.*
- 결과: github.com/Taewoo-Won/mini-ledger (Public), `[new branch] HEAD -> main`, 잔디 추가.

오늘의 산출물

- ✓ **Account** 클래스 — `deposit` / `withdraw` 직접 구현, 컴파일 성공
- ✓ **mini-ledger repo** — 커밋 `9e671d6`, push 완료, 잔디
- 📄 공개 repo: github.com/Taewoo-Won/mini-ledger

다음 작업 — 증분 2 (Bank)

- 만들 것: **Bank** 클래스 — `Map<String, Account>` 로 여러 계좌 관리 + `createAccount` / `getAccount` / `transfer`
- 읽을 것: 점프 투 자바 03-07 리스트(75–84), 03-08 맵(85–89) — 제네릭도 여기
- 포인트: `transfer` 는 두 계좌 검증을 돈 옮기기 전에 (오늘의 "검증 먼저" 확장 — 출금 성공 후 입금 실패하면 돈 증발)

오늘 배운 핵심 개념 (한 줄 사전)

개념	한 줄
변수	값을 담는, 이름 붙은 상자 (<code>long balance = 1000;</code>)
클래스 / 객체	설계도 / 그걸로 찍어낸 실체
필드	객체가 가진 데이터 (id, owner, balance)
생성자	객체 만들 때 필드 채우는 특수 메서드
메서드	객체의 동작 (deposit, withdraw)
<code>private</code> / <code>final</code>	캡슐화 / 재할당 불가
<code>if</code> + <code>throw</code>	조건 검사 + 에러 던지기
<code>IllegalArgumentException</code>	잘못된 인자 (음수 금액)
<code>IllegalStateException</code>	상태가 안 됨 (잔액 부족)
"검증 먼저"	상태 바꾸기 전에 검사 (ACID 씨앗)
돈은 <code>long</code>	부동소수점 금지
셸 vs 에디터	터미널(명령 실행) vs nano(내용 편집)
cat heredoc	터미널에서 파일 통째 생성 (<code>cat > 파일 << 'EOF'</code>)
<code>gh repo create</code>	repo 생성 + 연결 + push 한 줄

한 줄 회고

어제는 *환경*, 오늘은 첫 *진짜 코드*. 제로베이스라 Account 코드조차 안 읽혀 막막했지만 — 변수부터 다시 밟고, **deposit/withdraw** 로직을 내 손으로 떠올려 짰다. AI가 짜주는 걸 거부하고 직접 쓴 게 핵심. *읽기에서 쓰기*로 넘어가는 첫 발을 뒀다.